



Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Ingeniería en ciencias de la computación



Computo paralelo y distribuido

2.55 Proyecto 2a. Evaluación

José Saul De Lira Miramontes

David Alejandro Pérez González 367759

Ana Sofía Ledezma Díaz 367897

Evelyn Oyuky Torres Alanís 367868

11/05/2026

Main.py

Es lo que comienza el sistema. Usa la interfaz MPI (**M**essage **P**assing **I**nterface) para dividir la base de datos en múltiples hilos de procesamiento. Se encarga de la lógica de sincronización, la recolección de resultados de los nodos y la generación del reporte final de integridad del sistema.

```
main.py > main
1 from mpi4py import MPI
2 import asyncio
3 import pandas as pd
4 import numpy as np
5 from ingest import get_all_data
6 from processing import clean_data, gpu_analysis
7
8 def main():
9     comm = MPI.COMM_WORLD
10    rank = comm.Get_rank()
11    size = comm.Get_size()
12
13    # Nodo Maestro se encarga de la ingestión y limpieza de datos
14    if rank == 0:
15        print("--- Iniciando Pipeline Geoespacial y MPI (Equipo 1) ---")
16        try:
17            raw_data = asyncio.run(get_all_data())
18            df_w, df_f = clean_data(raw_data.get("weather"), raw_data.get("flights"))
19
20            if df_f.empty:
21                print(" ERROR: No hay datos de vuelos disponibles.")
22                comm.Abort()
23
24            # Presión de Argentina para ajustar la sensibilidad de la detección de anomalías
25            valor_base = df_w['Presion'].iloc[-1] if not df_w.empty else 1013.25
26
27            # dividimos el DataFrame de vuelos en partes iguales para cada nodo
28            chunks = np.array_split(df_f, size)
29            cols = df_f.columns.tolist()
30        except Exception as e:
31            print(f" Fallo en Nodo Maestro: {e}")
32            comm.Abort()
33    else:
34        chunks, cols, valor_base = None, None, None
35
36    # Distribuimos los datos a los nodos trabajadores
37    my_chunk_raw = comm.scatter(chunks, root=0)
38    cols = comm.bcast(cols, root=0)
39    valor_base = comm.bcast(valor_base, root=0)
40
41    my_chunk = pd.DataFrame(my_chunk_raw, columns=cols) if isinstance(my_chunk_raw, np.ndarray) else my_chunk_raw
42
43    # Cada núcleo usa la RTX 3050 para calcular distancias (Haversine) y anomalías
44    local_results = gpu_analysis(my_chunk, valor_base)
```

```
44    local_results = gpu_analysis(my_chunk, valor_base)
45
46    # Recolectamos los resultados de todos los nodos al maestro
47    all_results = comm.gather(local_results, root=0)
48
49    # Reporte final en el nodo maestro
50    if rank == 0:
51        print("\n" + "="*60)
52        print("--- SISTEMA DE VIGILANCIA: ANOMALIAS DE ALTITUD (EQUIPO 1) ---")
53        print("="*60)
54
55        # Unificación de resultados de los nodos MPI
56        vuelos_totales = pd.concat([pd.DataFrame(r['data']) for r in all_results if 'data' in r])
57        total_anomalias = int(vuelos_totales['es_anomalia'].sum())
58
59        if 'distancia_al_sensor_km' in vuelos_totales.columns:
60            vuelos_cerca = len(vuelos_totales[vuelos_totales['distancia_al_sensor_km'] < 1500])
61        else:
62            vuelos_cerca = 0
63
64        print(f" -> Aeronaves Globales en Radar: {len(vuelos_totales)}")
65        print(f" -> Aeronaves en Geocerca de Influencia (<1500km): {vuelos_cerca}")
66        print(f" -> Telemetria Barometrica Local (ThingSpeak): {valor_base:.2f} hPa")
67        print(f" -> Alertas por Maniobras de Altitud Atípicas: {total_anomalias}")
68
69        if total_anomalias > 0:
70            print(f" [AVISO] Se detectaron {total_anomalias} vuelos con perfiles de altitud fuera de rango.")
71        else:
72            print(" [ESTADO] Operaciones Normales: Perfiles de vuelo estables.")
73
74        print("="*60)
75
76        # Exportación para el Dashboard de Streamlit
77        vuelos_totales['presion_servicio'] = valor_base
78        vuelos_totales.to_csv("resultados_finales.csv", index=False)
79        print("\n[OK] Pipeline finalizado. Resultados exportados a CSV.")
80
81    if __name__ == "__main__":
82        main()
```

Processing.py

Aquí procesamos todos los datos, utiliza la librería CuPy para ejecutar cálculos vectorizados directamente en los núcleos CUDA de la GPU. Representa el núcleo de procesamiento de alto rendimiento

```
processing.py > clean_data
1  import pandas as pd
2  import numpy as np
3  try:
4      import cupy as cp
5  except ImportError:
6      cp = np
7
8  def clean_data(weather_raw, flights_raw):
9      df_w = pd.DataFrame()
10     if isinstance(weather_raw, dict) and 'feeds' in weather_raw:
11         df_w = pd.DataFrame(weather_raw['feeds'])
12         if 'field3' in df_w.columns:
13             df_w['Presion'] = pd.to_numeric(df_w['field3'], errors='coerce')
14             df_w = df_w.dropna(subset=['Presion'])
15
16     df_f = pd.DataFrame()
17     if isinstance(flights_raw, dict) and 'response' in flights_raw:
18         df_f = pd.DataFrame(flights_raw['response'])
19         for col in ['delay', 'speed', 'alt', 'lat', 'lng']:
20             if col in df_f.columns:
21                 df_f[col] = pd.to_numeric(df_f[col], errors='coerce').fillna(0)
22             else:
23                 df_f[col] = 0.0
24     return df_w, df_f
25
26
27 # Coordenadas exactas de tu sensor en Argentina (Ej: Buenos Aires)
28 LAT_SENSOR = -42.7859055676
29 LON_SENSOR = -64.9964695031
30
31 # Función de análisis en la GPU usando CuPy para cálculos masivos de distancia y detección de anomalías
32 def gpu_analysis(df, presion_base):
33     """Análisis Geoespacial y Detección de Anomalías de Altitud por Clima"""
34     if df.empty: return {"data": df.to_dict()}
35
36     try:
37         # analizamos la ALTITUD (alt) para detectar maniobras evasivas
38         altitudes = cp.array(df['alt'].astype(float).values)
39         velocidades = cp.array(df['speed'].astype(float).values) # Solo para guardarla
40         lat_vuelos = cp.array(df['lat'].astype(float).values)
41         lon_vuelos = cp.array(df['lng'].astype(float).values)
42
43         # 2. CÁLCULO MÁSIVO DE DISTANCIAS (Haversine Formula en CUDA)
44         lat1, lon1 = cp.radians(LAT_SENSOR), cp.radians(LON_SENSOR)
```

```

45     lat2, lon2 = cp.radians(lat_vuelos), cp.radians(lon_vuelos)
46
47     dlat = lat2 - lat1
48     dlon = lon2 - lon1
49
50     a = cp.sin(dlat/2)**2 + cp.cos(lat1) * cp.cos(lat2) * cp.sin(dlon/2)**2
51     c = 2 * cp.arcsin(cp.sqrt(a))
52     radio_tierra = 6371
53     distancias_km = c * radio_tierra
54
55     # 3. NIVEL DE RAREZA DE ALTITUD Y UMBRAL DINÁMICO
56     mean_a = cp.mean(altitudes)
57     std_a = cp.std(altitudes) + 1e-5
58     z_scores = cp.abs((altitudes - mean_a) / std_a)
59
60     # Ajustamos el umbral de detección según la presión atmosférica
61     # si tenemos: Baja presión = Tormentas = Detectamos cambios de altitud atípicos en la proximidad del sensor
62     if presion_base < 1010:
63         umbral = cp.where(distancias_km < 1500, 1.5, 2.5)
64     else: #si tenemos un Clima estable = Tolerancia normal de vuelo
65         umbral = cp.where(distancias_km < 1500, 2.0, 2.5)
66
67     anomalias = cp.where(z_scores > umbral, 1, 0)
68
69     # Guardamos nuestros resultados
70     df['distancia_al_sensor_km'] = cp.asnumpy(distancias_km)
71     df['z_score'] = cp.asnumpy(z_scores)
72     df['es_anomalia'] = cp.asnumpy(anomalias)
73     df['presion_vuelo'] = cp.asnumpy(presion_base * cp.power((1 - 0.000068753 * altitudes), 5.2559))
74
75 except Exception as e:
76     print(f"Error en Kernel GPU: {e}")
77
78 return {"data": df.to_dict()}
79
80

```

Ingest.py

Aquí tomamos los datos de las páginas externas. Utilizamos programación asíncrona con Asyncio para realizar peticiones a la API de AirLabs y a ThingSpeak. Su función principal es normalizar y limpiar los datos

```
ingest.py > get_all_data
1  import aiohttp
2  import asyncio
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  async def fetch_api(session, url):
9      async with session.get(url) as response:
10         if response.status == 200:
11             return await response.json()
12         return {"error": f"Status {response.status}"}
13
14  async def get_all_data():
15      ts_id = os.getenv("THINGSPEAK_CHANNEL_ID")
16      al_key = os.getenv("AIRLABS_API_KEY")
17
18      urls = {
19          "weather": f"https://api.thingspeak.com/channels/{ts_id}/feeds.json?results=50",
20          "flights": f"https://airlabs.co/api/v9/flights?api_key={al_key}"
21      }
22
23      async with aiohttp.ClientSession() as session:
24          # Creamos las tareas
25          names = list(urls.keys())
26          tasks = [fetch_api(session, urls[name]) for name in names]
27
28          # Esperamos los resultados de las solicitudes a las APIs
29          results = await asyncio.gather(*tasks)
30
31          # Devolvemos un diccionario con los resultados de cada API
32          return dict(zip(names, results))
```

Dashboard.py

Es la interfaz gráfica desarrollada en Streamlit para la visualización de datos en tiempo real. Procesa el archivo de salida para representar los indicadores clave de rendimiento. Permite la auditoría visual de los vuelos marcados como anomalías críticas basándose en el contexto ambiental.

```
dashboard.py > ...
1  import streamlit as st
2  import pandas as pd
3  import time
4  import os
5
6  st.set_page_config(page_title="PROYECTO 2 - EQUIPO 1", layout="wide")
7
8  def load_data():
9      if os.path.exists("resultados_finales.csv"):
10         try:
11             df = pd.read_csv("resultados_finales.csv")
12             df_map = df.dropna(subset=['lat', 'lng']).copy()
13             df_map = df_map.rename(columns={'lng': 'lon'})
14             return df_map
15         except: return None
16     return None
17
18 st.title(" Radar Inteligente: Detección de Anomalías Geoespaciales")
19 st.markdown("**Arquitectura HPC: MPI + CUDA | Variable de Control: Presión Barométrica (IoT)**")
20 st.markdown("----")
21
22 placeholder = st.empty()
23
24 while True:
25     df = load_data()
26     if df is not None and not df.empty:
27         with placeholder.container():
28             # 1. KPIs de Control
29             k1, k2, k3, k4 = st.columns(4)
30
31             p_base = df['presion_servicio'].iloc[0] if 'presion_servicio' in df.columns else 1013.25
32             total_anomalias = df['es_anomalia'].sum() if 'es_anomalia' in df.columns else 0
33
34             # Filtro para la distancia de 1,500 km
35             df_distancia = df[df['distancia_al_sensor_km'] < 1500]
36             vuelos_cerca = len(df_distancia)
37
38             k1.metric("Vuelos en Radar", f"{len(df):,}")
39             k2.metric("En distancia (<1500km)", f"{vuelos_cerca}")
40             k3.metric("Referencia IoT (hPa)", f"{p_base:.2f}")
41             k4.metric("Alertas de Altitud", f"{int(total_anomalias)}", delta="Análisis GPU", delta_color="inverse")
42
43             #Visual
44             col_map, col_graph = st.columns([2, 1])
45             with col_map:
46                 with col_map:
47                     st.subheader(" Monitoreo Global y Proximidad")
48                     st.map(df[['lat', 'lon']].head(200))
49
50             with col_graph:
51                 st.subheader(" Nivel de rareza De Altitud (Vuelos Cercanos)")
52                 if 'z_score' in df_distancia.columns:
53                     # Grafica de vuelos dentro del filtro de la distancia
54                     if not df_distancia.empty:
55                         st.bar_chart(df_distancia['z_score'].head(50))
56                     else:
57                         st.warning("No hay aeronaves detectadas en el radio de 1,500km.")
58
59                 st.subheader(" Detalle de Aeronaves en Zona de Influencia")
60                 cols_show = ['flight_iata', 'distancia_al_sensor_km', 'alt', 'speed', 'z_score', 'es_anomalia']
61                 # Tabla de datos con los vuelos dentro del filtro de la distancia
62                 df_show = df_distancia[cols_show].sort_values(by='distancia_al_sensor_km').head(50)
63
64                 df_show = df_show.rename(columns={'z_score': 'Nivel de rareza'})
65                 st.dataframe(df_show, width='stretch')
66
67                 st.info(f"Actualización: {time.strftime('%H:%M:%S')}. La gráfica de Z-Score solo muestra aeronaves dentro del radio de influencia.")
68
69     time.sleep(5)
```

Ejecución del programa

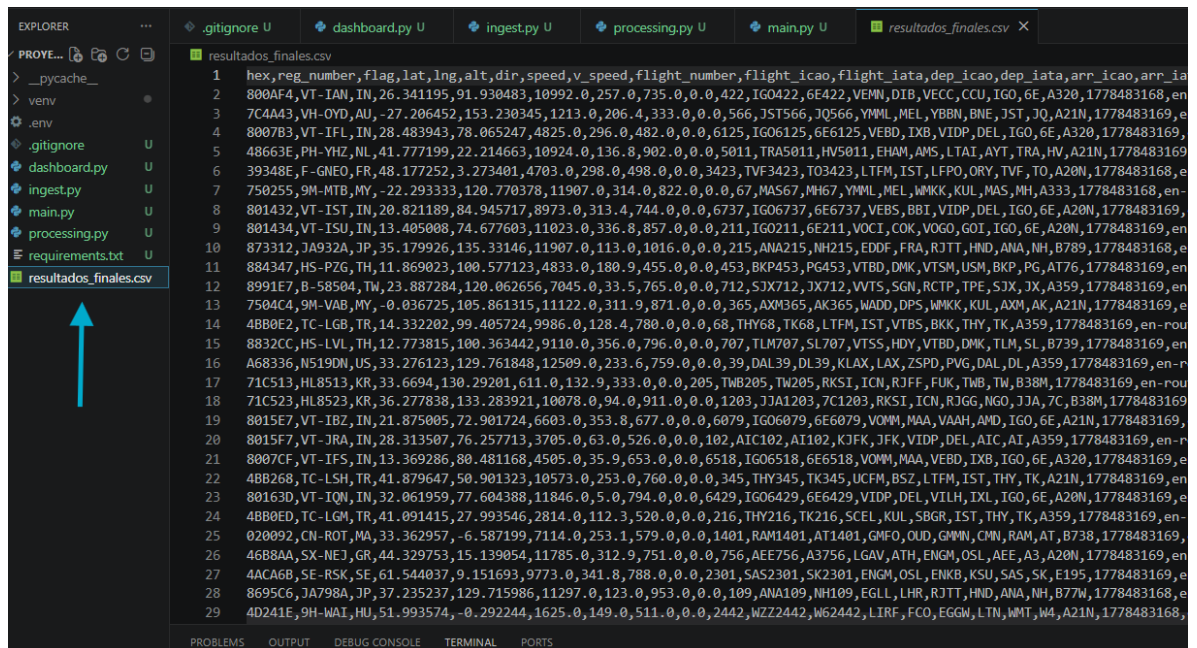
1. Primero ejecutamos `mpiexec -n 4 python main.py`, Este comando divide la carga de trabajo activando 4 núcleos del procesador al mismo tiempo. Uno funciona como coordinador y reparte los miles de vuelos entre los otros tres para que los analicen en equipo, logrando que el programa termine muchísimo más rápido que si lo hiciera paso a paso.

```
(venv) PS C:\Users\david\Desktop\Proyecto2> mpiexec -n 4 python main.py
--- Iniciando Pipeline Geoespacial y MPI (Equipo 1) ---

=====
--- SISTEMA DE VIGILANCIA: ANOMALIAS DE ALTITUD (EQUIPO 1) ---
=====
-> Aeronaves Globales en Radar: 5611
-> Aeronaves en Geocerca de Influencia (<1500km): 23
-> Presion Atmosferica Local (ThingSpeak): 1012.75 hPa
-> Alertas por Maniobras de Altitud Atipicas: 1
[AVISO] Se detectaron 1 vuelos con perfiles de altitud fuera de rango.
=====

[OK] Pipeline finalizado. Resultados exportados a CSV.
```

Ya con los datos que nos arroja el programa (el cual lo hace increíblemente rápido) estos se guardaran en un .CSV



```
EXPLORER  ...
PROVE...  [Icons]
> __pycache__
> venv
.env
.gitignore
dashboard.py
ingest.py
main.py
processing.py
requirements.txt
resultados_finales.csv

resultados_finales.csv
1 hex,reg_number,flag,lat,lng,alt,dir,speed,v_speed,flight_number,flight_icao,flight_iata,dep_icao,dep_iata,arr_icao,arr_iata
2 800AF4,VT- IAN, IN, 26.341195, 91.930483, 10992.0, 257.0, 735.0, 0.0, 422, IGO422, 6E422, VEMN, DIB, VECC, CCU, IGO, 6E, A320, 1778483168, en-
3 7C4A43,VH-OYD, AU, -27.206452, 153.230345, 1213.0, 206.4, 333.0, 0.0, 566, JST566, JQ566, YMML, MEL, YBBN, BNE, JST, JQ, A21N, 1778483169, e
4 8007B3,VT- IFL, IN, 28.483943, 78.065247, 4825.0, 296.0, 482.0, 0.0, 6125, IGO6125, 6E6125, VEBD, IXB, VIDP, DEL, IGO, 6E, A320, 1778483169, e
5 48663E,PH-YHZ, NL, 41.777199, 22.214663, 10924.0, 136.8, 902.0, 0.0, 5011, TRA5011, HV5011, EHAM, AMS, LTAI, AYT, TRA, HV, A21N, 1778483169, e
6 39348E, F-GNEO, FR, 48.177252, 3.273401, 4703.0, 298.0, 498.0, 0.0, 3423, TVF3423, TO3423, LTFM, IST, LFPO, ORY, TVF, TO, A20N, 1778483168, e
7 750255,9M-MTB, MY, -22.293333, 120.770378, 11907.0, 314.0, 822.0, 0.0, 67, MAS67, MH67, YMML, MEL, WMKK, KUL, MAS, MH, A333, 1778483168, en-
8 801432,VT- IST, IN, 20.821189, 84.945717, 8973.0, 313.4, 744.0, 0.0, 6737, IGO6737, 6E6737, VEB5, BBI, VIDP, DEL, IGO, 6E, A20N, 1778483169, e
9 801434,VT- ISU, IN, 13.405008, 74.677603, 11023.0, 336.8, 857.0, 0.0, 211, IGO211, 6E211, VOIC, COK, VOGO, GOI, IGO, 6E, A20N, 1778483169, en-
10 873312, JA932A, JP, 35.179926, 135.33146, 11907.0, 113.0, 1016.0, 0.0, 215, ANA215, NH215, EDDF, FRA, RJTT, HND, ANA, NH, B789, 1778483168, e
11 884347, HS-PZG, TH, 11.869023, 100.577123, 4833.0, 180.9, 455.0, 0.0, 453, BKP453, PG453, VTBD, DMK, VTSM, USM, BKP, PG, AT76, 1778483169, e
12 8991E7, B-58504, TW, 23.887284, 120.062656, 7045.0, 33.5, 765.0, 0.0, 712, SJX712, JX712, WTS, SGN, RCTP, TPE, SJX, JX, A359, 1778483169, en-
13 7504C4, 9M-VAB, MY, -0.036725, 105.861315, 11122.0, 311.9, 871.0, 0.0, 365, AXM365, AK365, WADD, DPS, WMKK, KUL, AXM, AK, A21N, 1778483169, e
14 4BB0E2, TC-LGB, TR, 14.332202, 99.405724, 9986.0, 128.4, 780.0, 0.0, 68, THY68, TK68, LTFM, IST, VTBS, BKK, THY, TK, A359, 1778483169, en-rou
15 8832CC, HS-LVL, TH, 12.773815, 100.363442, 9110.0, 356.0, 796.0, 0.0, 707, TLM707, SL707, VTSS, HDY, VTBD, DMK, TLM, SL, B739, 1778483169, en-
16 A68336, N519DN, US, 33.276123, 129.761848, 12509.0, 233.6, 759.0, 0.0, 39, DAL39, DL39, KLAX, LAX, ZSPD, PVG, DAL, DL, A359, 1778483169, en-rou
17 71C513, HL8513, KR, 33.6694, 130.29201, 611.0, 132.9, 333.0, 0.0, 205, TMB205, TW205, RKSI, ICN, RJFF, FUK, TWB, TW, B38M, 1778483169, en-rou
18 71C523, HL8523, KR, 36.277838, 133.283921, 10078.0, 94.0, 911.0, 0.0, 1203, JJA1203, 7C1203, RKSI, ICN, RJGG, NGO, JJA, 7C, B38M, 1778483169, e
19 8015E7, VT- IBZ, IN, 21.875005, 72.901724, 6603.0, 353.8, 677.0, 0.0, 6079, IGO6079, 6E6079, VOMM, MAA, VAAH, AMD, IGO, 6E, A21N, 1778483169, e
20 8015F7, VT- JRA, IN, 28.313507, 76.257713, 3705.0, 63.0, 526.0, 0.0, 102, AIC102, AI102, KJFK, JFK, VIDP, DEL, AIC, AI, A359, 1778483169, en-rou
21 8007CF, VT- IFS, IN, 13.369286, 80.481168, 4505.0, 35.9, 653.0, 0.0, 6518, IGO6518, 6E6518, VOMM, MAA, VEBD, IXB, IGO, 6E, A320, 1778483169, e
22 4BB268, TC- LSH, TR, 41.879647, 50.901323, 10573.0, 253.0, 760.0, 0.0, 345, THY345, TK345, UCFM, BSZ, LTFM, IST, THY, TK, A21N, 1778483169, en-rou
23 80163D, VT- IQN, IN, 32.061959, 77.604388, 11846.0, 5.0, 794.0, 0.0, 6429, IGO6429, 6E6429, VIDP, DEL, VILH, IXL, IGO, 6E, A20N, 1778483169, e
24 4BB0ED, TC- LGM, TR, 41.091415, 27.993546, 2814.0, 112.3, 520.0, 0.0, 216, THY216, TK216, SCEL, KUL, SBGR, IST, THY, TK, A359, 1778483169, en-rou
25 020092, CN-ROT, MA, 33.362957, -6.587199, 7114.0, 253.1, 579.0, 0.0, 1401, RAM1401, AT1401, GMFO, OUD, GMMN, CMN, RAM, AT, B738, 1778483169, e
26 46B8AA, SX-NEJ, GR, 44.329753, 15.139054, 11785.0, 312.9, 751.0, 0.0, 756, AEE756, A3756, LGAV, ATH, ENGM, OSI, AEE, A3, A20N, 1778483169, e
27 4ACA6B, SE- RSK, SE, 61.544037, 9.151693, 9773.0, 341.8, 788.0, 0.0, 2301, SAS2301, SK2301, ENGM, OSI, ENKB, KSU, SAS, SK, E195, 1778483169, e
28 8695C6, JA798A, JP, 37.235237, 129.715986, 11297.0, 123.0, 953.0, 0.0, 109, ANA109, NH109, EGLL, LHR, RJTT, HND, ANA, NH, B77W, 1778483168, e
29 4D241E, 9H-WAI, HU, 51.993574, -0.292244, 1625.0, 149.0, 511.0, 0.0, 2442, WZZ2442, W62442, LIRF, FCO, EGGW, LTN, WMT, W4, A21N, 1778483168, e
```

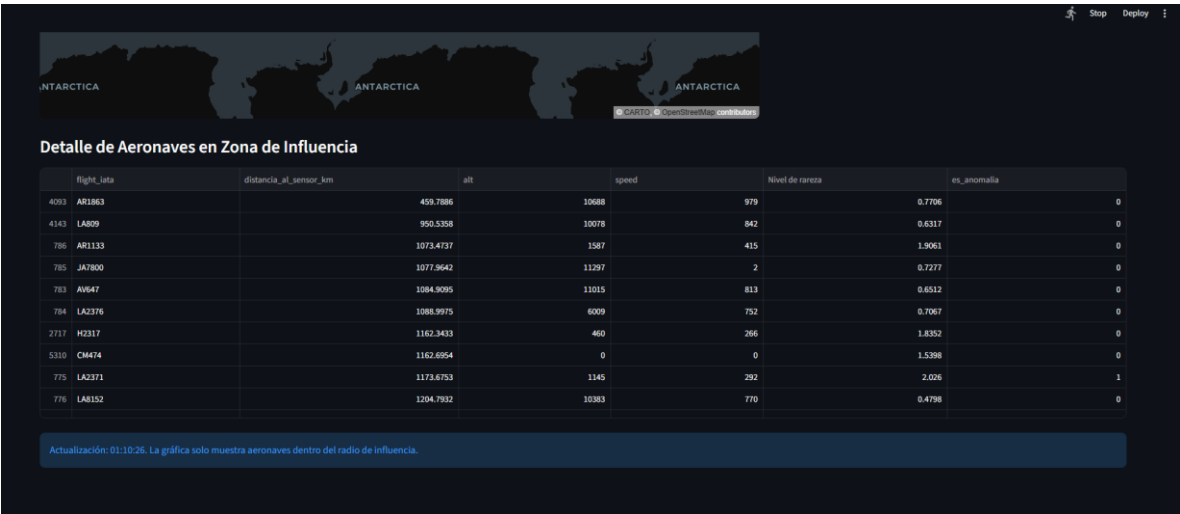
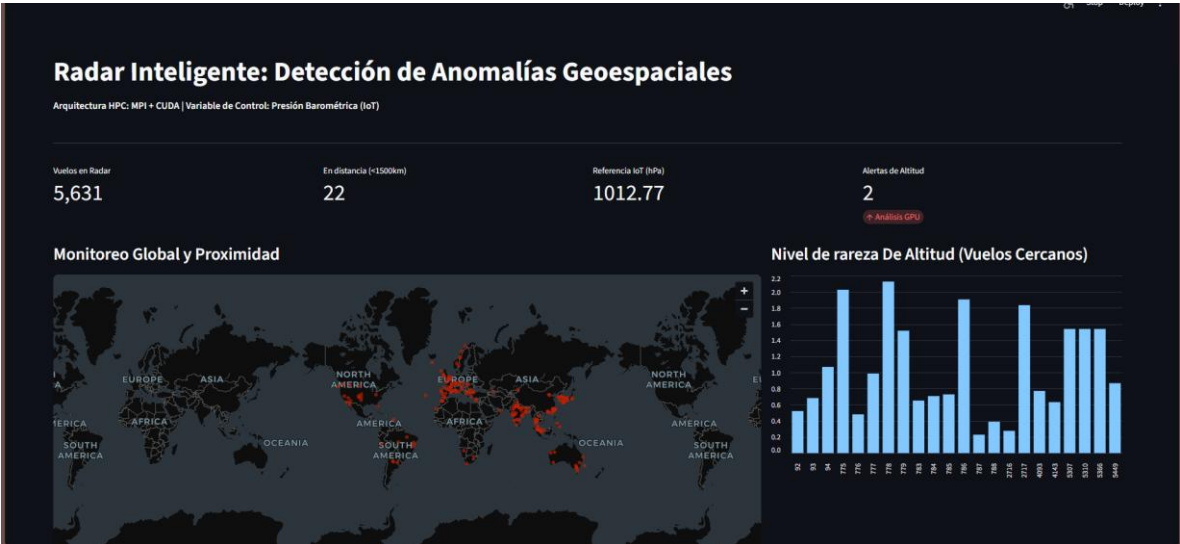
2. Con esto podemos ejecutar el siguiente comando: `streamlit run dashboard.py`, el cual ejecutara una pagina web con nuestro dashboard que creara todo el entorno visual sobre nuestro reporte

```
(venv) PS C:\Users\david\Desktop\Proyecto2> streamlit run dashboard.py
2026-05-11 01:08:15.754 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.100.100:8501
```

Al entrar al link de localhost podremos ver todo nuestro dashboard con la información actual que acabamos de reunir



Todo esto hecho con Streamlit